R1.04 — Architecture des Systèmes d'Exploitation : Gestion, Filiation et Signaux des Processus Linux

Thème principal : Gestion dynamique des processus, cycle de vie (Élu, Prêt, Bloqué), filiation parent-enfant et communication par signaux POSIX

Synthèse exécutive : Ce cours fondamental de R1.04 explicite le passage d'un programme statique (stocké sur disque) à un processus dynamique (actif en mémoire RAM). Il détaille l'ordonnancement multitâche préemptif par quanta de temps CPU, la structure de contrôle PCB (Process Control Block), l'arborescence généalogique issue du processus racine systemd (PID 1), le mécanisme de création fork()/exec(), ainsi que la gestion de l'exécution et des interruptions via les signaux POSIX.

1. Du Programme Statique au Processus Dynamique : Attributs, Multitâche et Cycle de Vie

Pour exécuter des instructions, un système d'exploitation convertit un programme (code exécutable inactif sur disque) en un processus (entité dynamique en mémoire RAM). Dans un système Unix/Linux, la simultanéité perçue repose sur le multitâche préemptif : le noyau n'attend pas la fin d'un processus pour passer au suivant, il reprend le contrôle dès l'expiration du quantum de temps CPU alloué. Le cycle de vie d'un processus traverse trois états fondamentaux : 1. **Élu (Running)** : Le processus occupe un cœur processeur et exécute ses instructions binaire. 2. **Prêt (Ready)** : Le processus dispose de toutes ses ressources en RAM et attend simplement que l'ordonnanceur lui attribue une tranche de temps processeur. 3. **Bloqué (Waiting/Blocked)** : Le processus s'interrompt pour attendre une donnée externe (lecture disque, saisie clavier, paquet réseau). Il ne consomme alors aucune puissance CPU. Dès que la donnée est disponible, le noyau bascule le processus à l'état **Prêt** (et jamais directement à l'état Élu).

- • Programme vs Processus : Un programme est un fichier binaire passif stocké sur disque ; un processus est une instance dynamique active chargée en mémoire RAM avec des ressources allouées.
- • Multitâche Préemptif & Ordonnanceur : Le noyau découpe le temps CPU en tranches appelées quanta. L'ordonnanceur (scheduler) interrompt de force les processus pour faire tourner les tâches sur le processeur.
- • Process Control Block (PCB) : Structure de données interne au noyau sauvegardant l'état CPU, le pointeur de pile, le tas, les registres, le PID, PPID, UID, la priorité et les descripteurs de fichiers ouverts.
- • Modèle des 3 États (Élu, Prêt, Bloqué) : Élu (exécution sur le CPU), Prêt (chargé en RAM et attend du temps CPU), Bloqué (suspendu en attente d'une opération d'Entrée/Sortie ou d'un événement). Un processus bloqué repasse obligatoirement par l'état Prêt avant de revenir Élu.

Cas concret enseignant : *L'analogie du livre de recettes : Le programme sur disque est un livre de recettes rangé dans la bibliothèque (passif, 0 électricité, 0 ressource). Le processus est le cuisinier qui réalise la recette dans sa cuisine (dynamique, occupant le plan de travail en RAM, consommant du temps de cuisson/CPU et manipulant des ustensiles équivalant aux descripteurs de fichiers).*

2. Arborescence et Généalogie des Processus : systemd, PID/PPID et fork()/exec()

Tout processus sous Linux s'inscrit dans un arbre généalogique strict dont la racine absolue est `systemd` (ou `init`), portant le PID 1. Aucun processus ne peut apparaître spontanément ex nihilo. La création d'un processus se déroule en deux temps grâce aux appels système `fork()` et `exec()` : 1. `fork()` : Le processus parent (ex: le shell Bash) demande au noyau de dupliquer son espace mémoire à l'identique. Une nouvelle entrée est créée dans la table des processus avec un nouveau PID, et le PPID du fils pointe sur le parent. 2. `exec()` (ou `execve()`) : Le nouveau processus enfant utilise cette fonction pour écraser son propre espace mémoire et charger les instructions d'un nouvel exécutable (ex: `/bin/ls`).

• Identifiants PID et PPID : Chaque processus possède un identifiant unique (PID - Process ID) et connaît le PID de son créateur parent (PPID - Parent Process ID).

• Processus Racine (systemd / init) : Premier processus espace utilisateur démarré par le noyau au boot, portant impérativement le PID 1.

• Duo d'appels système fork() et exec() : fork() duplique le processus parent pour créer un processus enfant identique ; exec() remplace le code du fils par celui du nouveau programme à exécuter.

Cas concret enseignant : *La division cellulaire (mitose) : fork() équivaut à la duplication d'une cellule parente pour former une cellule enfant identique. exec() correspond au moment où la cellule enfant change son ADN interne pour exécuter une tâche totalement différente.*

3. Communication Inter-Processus et Contrôle par Signaux POSIX

Les signaux POSIX constituent un mécanisme de communication asynchrone rudimentaire mais essentiel. À la réception d'un signal, un processus peut exécuter le comportement par défaut (souvent la fin du processus), ignorer le signal, ou l'intercepter via un gestionnaire dédié (Signal Handler). Il existe une distinction capitale entre les signaux doux et les signaux absolus : - **Signaux interceptables** : SIGINT (2) et SIGTERM (15) peuvent être capturés par le programmeur pour exécuter des nettoyages pré-fermeture (ex: sauvegarder un document ouvert). - **Signaux d'exception ininterceptables** : SIGKILL (9) et SIGSTOP (19) sont directement pris en charge par le noyau Linux. Le programme cible n'a aucun moyen de les intercepter ou d'empêcher sa propre destruction.

• Signal POSIX : Notification asynchrone ou interruption logicielle envoyée par le noyau ou un utilisateur à un processus.

• Gestionnaire de signal (Signal Handler) : Routine applicative personnalisée permettant de capturer (catch) un signal au lieu de subir l'action système par défaut.

• SIGINT (Signal 2) : Envoyé via 'Ctrl + C' pour demander l'interruption douce d'une commande interactive.

• SIGTERM (Signal 15) : Signal d'arrêt propre envoyé par défaut par 'kill '. Permet au processus de fermer ses fichiers et libérer ses ressources.

• SIGKILL (Signal 9) : Ordre de destruction inconditionnel exécuté par le noyau. Impossible à intercepter, ignorer ou masquer.

• SIGSTOP (19) et SIGCONT (18) : Mise en pause d'une tâche (Ctrl + Z) et reprise de son exécution au premier ou arrière-plan ('fg' / 'bg').

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de ranger vos affaires et de sortir (vous pouvez sauvegarder et fermer proprement). SIGKILL (9) est l'équipe d'intervention qui enfonce la porte et vous évacue sur-le-champ sans vous laisser toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Le tiret '-' devant le chiffre est indispensable car il indique à l'utilitaire 'kill' que la valeur 9 correspond à l'option désignant le numéro du signal (SIGKILL). Sans le tiret, la commande interprète '9' et '1234' comme étant deux identifiants de processus (PIDs) distincts, et tente alors d'envoyer le signal par défaut SIGTERM (15) au processus 9 et au processus 1234.

Question étudiant : Qu'est-ce qu'un processus 'Zombie' et comment peut-on le détruire à l'aide de la commande kill ?
Réponse du professeur : Un processus Zombie est une tâche dont l'exécution est déjà terminée, mais dont la ligne persiste dans la table des processus car son parent n'a pas encore lu son code de retour (via l'appel système wait()). Comme un Zombie est DÉJÀ mort, il n'exécute aucun code et la commande 'kill -9 ' n'a aucun effet. Pour faire disparaître un Zombie, il faut forcer le processus PARENT à lire l'état du fils, ou tuer le parent pour que le processus 'systemd' (PID 1) adopte l'orphelin et nettoie immédiatement la table des processus.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un administrateur effectue la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Expliquez le rôle exact du symbole '&', détaillez ce qui se passe lors de l'exécution de la commande kill -15 et décrivez le résultat de la vérification par ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé en fin de ligne indique au shell d'exécuter la commande 'sleep 600' en arrière-plan (background). Le terminal affiche le numéro de job (ex: [1]) et le PID attribué (ex: 4521), puis rend immédiatement la main à l'utilisateur sans bloquer le shell. Étape 2 : La commande 'ps -ef | grep sleep' filtre la liste globale des processus pour extraire la ligne correspondant à l'exécutable 'sleep'. Cela permet d'obtenir son PID exact (4521) ainsi que son PPID (le PID du shell interactif). Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM (Signal 15) au processus target. N'ayant pas de gestionnaire de signal spécifique détournant ce comportement, le processus 'sleep' exécute l'action système par défaut : la fermeture propre et la libération complète de sa mémoire en RAM.

Résultat : Le processus 'sleep 600' est arrêté de manière propre et ordonnée. Une seconde exécution de la commande 'ps -ef | grep sleep' confirme sa suppression définitive de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : Il s'agit du processus 'systemd' (ou 'init' sur d'anciens systèmes), qui porte impérativement le PID 1.

Q : Quel signal POSIX est transmis à un processus interactif lorsqu'un utilisateur tape le raccourci 'Ctrl + C' dans son terminal ?

R : Il s'agit du signal SIGINT (Signal n°2), qui demande l'interruption douce de l'exécution.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être capturé par un gestionnaire de signal écrit par un développeur ?

R : Parce que SIGKILL n'est pas transmis au code de l'application : il est directement intercepté et exécuté par le noyau Linux lui-même, qui détruit immédiatement le processus pour des raisons de sécurité et de contrôle système.